Lecture 13: Abstracting over behavior

Defining function objects to abstract over behavior

Related files:
BostonMarathon.java

Video Lessons

- Lesson 1 - Introduction to function objects part 1
- Lesson 2 - Introduction to function objects part 2

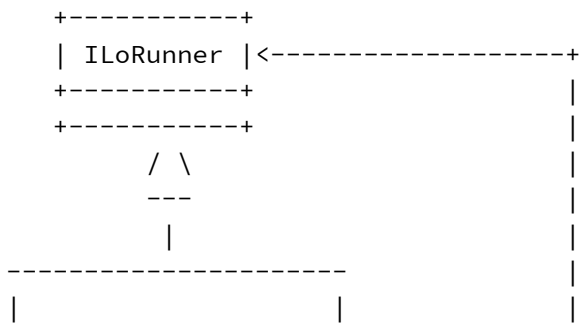
Introduction: Young at Heart — The Boston Marathon

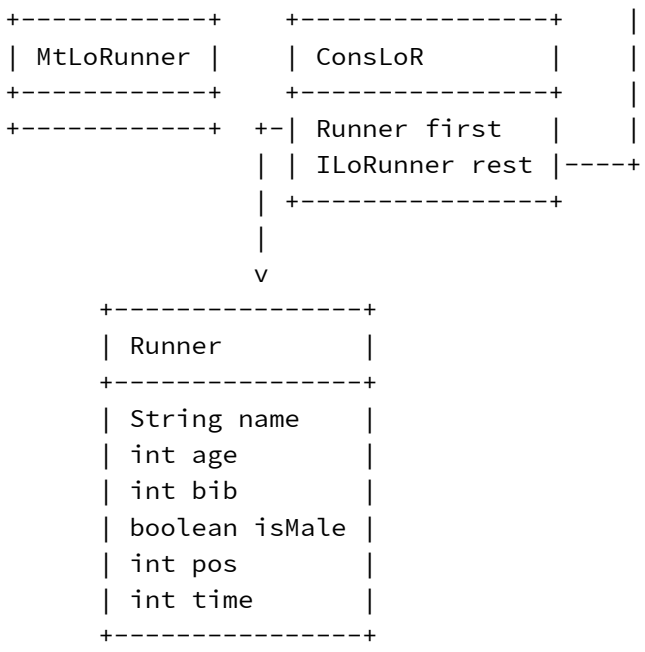
The most popular Boston sports event takes place every year on a Monday in April, on the Patriot's Day holiday. It is not a baseball game, or a football game. There are over 10000 athletes who take part in the event, with over 100000 spectators, bringing the whole city to a standstill for most of the day.

The Boston Marathon has been held for over 100 years, with some of the runners becoming legends. The best known among them is Johnny Kelly, who died in 2004 after having run the marathon over than fifty times—and winning it four times. Marathon officials commissioned a statue of 27-year-old Johnny Kelley (at his first victory) shaking hands with his 80-year-old self, entitled "Young at Heart".

Our challenge today is to help the organizers to keep track of all the runners. For each runner we need to record the name, the age, the bib number, the runner's time (in minutes), and whether the runner is male or female. We'll also record the starting position of each runner. The marathon itself can be represented by a list of runners.

The following classes represent the list of runners.





For today, we are simply going to look at various groups of runners. We'd like to find out all the runners who are male, and all the runners who are female. We'd like to find all the runners who start in the pack of the first 50 runners. We'd like to find all runners who finish the race in under four hours. We'd like to find all runners younger than age 40. Later we'll ask more complicated questions, too.

13.1 Warmup: answering the first few questions

We start with examples of runners and lists of runners:

```
// In Examples class
Runner johnny = new Runner("Kelly", 97, 999, true, 30, 360);
Runner frank  = new Runner("Shorter", 32, 888, true, 245, 130);
Runner bill = new Runner("Rogers", 36, 777, true, 119, 129);
Runner joan = new Runner("Benoit", 29, 444, false, 18, 155);

ILoRunner mtlist = new MTLoRunner();
ILoRunner list1 = new ConsLoRunner(johnny, new ConsLoRunner(joan, mtlist));
ILoRunner list2 = new ConsLoRunner(frank, new ConsLoRunner(bill, list1));
```

Let's try the first two questions: finding the list of all male runners, and the list of all female runners.

> ***Do Now!***
>
> What method (or methods) will we need to define to achieve this? What classes or interfaces should we modify to do so?

Both of these questions clearly produce an `ILoRunner`, and presumably must process an `ILoRunner`, so it seems we must define methods in the `ILoRunner` interface to implement them. We can't get away with just one method, though, since the results are

different:

```
// In ILoRunner
ILoRunner findAllMaleRunners();
ILoRunner findAllFemaleRunners();
```

Implementing these should be straightforward:

```
// In MtLoRunner
public ILoRunner findAllMaleRunners() { return this; }
public ILoRunner findAllFemaleRunners() { return this; }
```

So far so good...

```
// In ConsLoRunner
public ILoRunner findAllMaleRunners() {
  if (this.first.isMale) {
    return new ConsLoRunner(this.first, this.rest.findAllMaleRunners());
  }
  else {
    return this.rest.findAllMaleRunners();
  }
}
public ILoRunner findAllFemaleRunners() {
  if (!this.first.isMale) {
    return new ConsLoRunner(this.first, this.rest.findAllFemaleRunners());
  }
  else {
    return this.rest.findAllFemaleRunners();
  }
}
```

Except this code violates the template for `ConsLoRunner` methods.

> ***Do Now!***
>
> How?

We're using a field-of-a-field access (`this.first.isMale`), which is not allowed. (We saw how in general, field-of-a-field access isn't even type-correct in Lecture 12: Defining sameness for complex data, part 2.) So we need to define a helper method in the `Runner` class:

```
// In Runner
public boolean isMaleRunner() { return this.isMale; }
```

And now we can rewrite our methods to use this helper instead.

```
// In ConsLoRunner
public ILoRunner findAllMaleRunners() {
  if (this.first.isMaleRunner()) {
    return new ConsLoRunner(this.first, this.rest.findAllMaleRunners());
```

```
    }
    else {
      return this.rest.findAllMaleRunners();
    }
  }
  public ILoRunner findAllFemaleRunners() {
    if (!this.first.isMaleRunner()) {
      return new ConsLoRunner(this.first, this.rest.findAllFemaleRunners());
    }
    else {
      return this.rest.findAllFemaleRunners();
    }
  }
```

Of course, no methods are complete without tests to confirm they work:

```
// In Examples class
boolean testFindMethods(Tester t) {
  return
    t.checkExpect(this.list2.findAllFemaleRunners(),
                  new ConsLoRunner(this.joan, new MtLoRunner())) &&
    t.checkExpect(this.list2.findAllMaleRunners(),
                  new ConsLoRunner(this.frank,
                    new ConsLoRunner(this.bill,
                      new ConsLoRunner(this.johnny, new MtLoRunner()))));
}
```

Let's try the next question: runners who start in the first 50 positions.

> ***Do Now!***
>
> Following the pattern above, design this method. What helpers are needed?

As with the previous two examples, the empty list just returns empty:

```
// In MtLoRunner
public ILoRunner findRunnersInFirst50() { return this; }
```

While the non-empty case uses an if-test:

```
// In ConsLoRunner
public ILoRunner findRunnersInFirst50() {
  if (this.first.posUnder50()) {
    return new ConsLoRunner(this.first, this.rest.findRunnersInFirst50());
  }
  else {
    return this.rest.findRunnersInFirst50();
  }
}
```

with a helper method on `Runner`:

```
// In Runner
boolean posUnder50() { return this.pos <= 50; }
```

This is getting tedious, and we still have several questions left unanswered!

13.2 Abstracting over behavior: Function objects

Looking at the definitions above, we can see a lot of repetitive code. Whenever we see such repetition, we know that the design recipe for abstraction tells us to find the parts of the code that differ, find the parts of the code that are the same, and separate the common parts of the code into a single shared implementation. Trying that here, we see the following common pattern:

```
// In MtLoRunner
public ILoRunner find...() { return this; }
```

```
// In ConsLoRunner
public ILoRunner find...() {
  if (this.first...) {
    return new ConsLoRunner(this.first, this.rest.find...());
  }
  else {
    return this.rest.find...();
  }
}
```

The signature of all our `find...` methods is the same, and the skeleton of the code is the same: the only parts that differ are the precise names of the `find...` methods and the precise condition we test on `this.first`. If we can abstract away that test, then we can consolidate all these definitions into just one method. But what abstraction can we use? Abstract classes won't help: they let us share field and method definitions, and we want different behaviors for this test. Inheritance won't help: we don't want to define subtypes of lists that can each answer just one question, but rather one kind of list that can answer multiple questions. Delegation might help...but how? We're already delegating to the `Runner` class, and cluttering its definition with lots of little helpers.

At this point, you should recognize this pattern from Fundies I: we need *higher-order functions*, where we can pass in the function to do the test on `Runner`s for us. But Java doesn't have *functions*: it only has classes and methods.

> ***Do Now!***
>
> Do you see a way around this problem? Think back to Assignment 2: Designing methods for complex data...

Look at the signatures for the helper methods we defined in the `Runner` class: they all operate on a `Runner` and produce a `boolean`. Suppose instead of defining these helper methods as methods *on the `Runner` class*, we defined them individually as methods *in helper classes*. Instead of having `this` be the `Runner`, we'll have these methods take a

`Runner` as a parameter:

```
class RunnerIsMale {
  boolean isMaleRunner(Runner r) { return r.isMale; }
}
class RunnerIsFemale {
  boolean isFemaleRunner(Runner r) { return !r.isMale; }
}
class RunnerIsInFirst50 {
  boolean isInFirst50(Runner r) { return r.pos <= 50; }
}
```

So far, not much improvement, but at least our `Runner` class has been restored to its original simplicity.

There's clearly room to improve this code: the method names are redundant with the class names. What should we call these methods? Well, what can we do with a `RunnerIsMale` object? We can just invoke its single method, applying it to some `Runner`. What can we do with a `RunnerIsFemale` object? We can invoke its single method, applying it to some `Runner`. Ditto for `RunnerIsInFirst50`. All we can do with these objects is *apply* their single method to a `Runner`. We might as well just name the method `apply`! And once we do that, we see that all three classes have *exactly* the same signature: we should recognize that by defining it as an interface!

```
interface IRunnerPredicate {
  boolean apply(Runner r);
}
class RunnerIsMale implements IRunnerPredicate {
  public boolean apply(Runner r) { return r.isMale; }
}
class RunnerIsFemale implements IRunnerPredicate {
  public boolean apply(Runner r) { return !r.isMale; }
}
class RunnerIsInFirst50 implements IRunnerPredicate {
  public boolean apply(Runner r) { return r.pos <= 50; }
}
```

We name the interface `IRunnerPredicate` because it describes objects that can answer a `boolean`-valued question (i.e., a *predicate*) on `Runner`s.

Now that we have constructed this `IRunnerPredicate` abstraction, we can use it to revise our `find...` methods: we can enhance them to take an `IRunnerPredicate` as a parameter, and delegate to it to answer the appropriate test on the elements of the list.

```
// In ILoRunner
ILoRunner find(IRunnerPredicate pred);
```

```
// In MtLoRunner
public ILoRunner find(IRunnerPredicate pred) { return this; }
```

```
// In ConsLoRunner
public ILoRunner find(IRunnerPredicate pred) {
```

```
    if (pred.apply(this.first)) {
      return new ConsLoRunner(this.first, this.rest.find(pred));
    }
    else {
      return this.rest.find(pred);
    }
  }
```

Notice that this is almost exactly the same definition as we had in Fundies I for the `(find ...)` function: we have abstracted the test into a parameter that we can use in the body of the `find` method.

> This is certainly less convenient than `lambda`, which let us define new, anonymous functions whenever and wherever they were needed. The latest versions of Java are now, *finally*, adding more convenient syntax to make defining lambdas easier...

In Java, these kinds of objects that are defined solely for the method contained inside them (that in Fundies I were simply *functions*) are called, naturally enough, *function objects*. We define an interface that describes the signature of the function we'd like to abstract, and define our original method to take a parameter of that type, which we then delegate to as needed within the method. Then to define new operations to be used with our method, all we need to do is define new classes that implement the interface.

To use these new function objects, we rewrite our tests:

```
// In Examples class
boolean testFindMethods(Tester t) {
  return
    t.checkExpect(this.list2.find(new RunnerIsFemale()),
                  new ConsLoRunner(this.joan, new MtLoRunner())) &&
    t.checkExpect(this.list2.find(new RunnerIsMale()),
                  new ConsLoRunner(this.frank,
                    new ConsLoRunner(this.bill,
                      new ConsLoRunner(this.johnny, new MtLoRunner()))));
}
```

> ***Do Now!***
>
> Design whatever helper methods or classes you need to solve the problem, "Find all runners who finish in under 4 hours."

All we need do is define a new class implementing `IRunnerPredicate`:

```
class FinishIn4Hours implements IRunnerPredicate {
  public boolean apply(Runner r) { return r.time < 240; }
}
```

```
// In Examples class
```

```
boolean testFindUnder4Hours(Tester t) {
  return
    t.checkExpect(this.list2.find(new FinishIn4Hours()),
                  new ConsLoRunner(this.frank,
                    new ConsLoRunner(this.bill,
                      new ConsLoRunner(this.joan, new MtLoRunner()))));
}
```

We don't have to modify the `Runner`, `MtLoRunner` and `ConsLoRunner` classes or the `ILoRunner` interface at all!

13.3 Compound questions

How might we find the list of all male runners who finish in under 4 hours? How might we find the list of all female runners younger than 40 who started in the first 50 starting positions? We could continue to define new `IRunnerPredicate` classes for each of these...but notice that we've already answered each of the component questions here. It would be a shame not to be able to reuse their implementations.

What does the `IRunnerPredicate` interface promise? It says that for any class that implements the interface, we can ask instances of that class a boolean question about a `Runner`—but it says nothing about how the class should implement the answer to that question. If we wanted, we could have a class that *delegates* answering the question to *other* `IRunnerPredicate`s!

> ***Do Now!***
>
> What logical operator is being used in the combined questions above?

We can define a new class, `AndPredicate`, as follows:

```
// Represents a predicate that is true whenever both of its component predicates are true
class AndPredicate implements IRunnerPredicate {
  IRunnerPredicate left, right;
  AndPredicate(IRunnerPredicate left, IRunnerPredicate right) {
    this.left = left;
    this.right = right;
  }
  public boolean apply(Runner r) {
    return this.left.apply(r) && this.right.apply(r);
  }
}
```

> ***Do Now!***
>
> Use this new class to answer the questions above.

```
// In Examples class
```

```
boolean testCombinedQuestions(Tester t) {
  return
    t.checkExpect(this.list2.find(
                    new AndPredicate(new RunnerIsMale(), new FinishIn4Hours())),
                  new ConsLoRunner(this.frank,
                    new ConsLoRunner(this.bill, new MtLoRunner())))  &&
    t.checkExpect(this.list2.find(
                    new AndPredicate(new RunnerIsFemale(),
                      new AndPredicate(new RunnerIsYounger40(),
                                       new RunnerIsInFirst50()))),
                  new ConsLoRunner(this.joan, new MtLoRunner()));
}
```

These kinds of function objects that are constructed with additional parameters are known as *parameterized function objects*, and they are both very useful and very common.

> ***Do Now!***
>
> Design an answer to the problem, “Find all runners who are female or who finish in less than 4 hours.”